



Classifying Deceleration Measurement Sequences

Nikita Biswas
Sebastian Garcia
Chearan Yoo
Daniel Useche Corchuelo

Technische Universität Dresden
Faculty of Mathematics

June 2025

Contents

1	Introduction	2
2	Task 1: FCN, 2D CNNs vs. LSTMs - Parameter Tests and Comparison	3
2.1	Exploratory Data Analysis (Train vs. Test data set)	3
2.1.1	Class Distribution	3
2.1.2	Impact of Physical Features on Signals	5
2.2	FCN	7
2.2.1	Background	7
2.2.2	Architecture	7
2.2.3	Analysis and Results	8
2.3	2D CNN	9
2.3.1	Background	9
2.3.2	Architecture	10
2.3.3	Result and Analysis	10
2.4	LSTM (Long Short-Term Memory)	11
2.4.1	Background	11
2.4.2	Architecture	12
2.4.3	Analysis	12
2.5	Best Network	13
3	Task 2: Bias in the dataset	15
3.1	Network's performance	15
3.1.1	Deceleration measurements	16
3.2	Data preprocessing	16
3.3	Data augmentation	17
3.4	Debiasing Variational Autoencoder	19
3.4.1	Model Architecture	20
3.4.2	Result and Analysis	20
4	Additional Task 2: Ensemble Classifiers	22
4.1	Binary Head Model	22
4.2	Physical Features Head Model	23
5	Conclusion	25

1 Introduction

FSD Fahrzeugssystemdaten GmbH is the central authority according to the German Road Traffic Act (StVG) for the further development of the main inspection, the innovative testing technologies required for it, and other requirements.

One of the main inspections is of the braking system, which plays a crucial role in the safety of a vehicle. For this, FSD GmbH has developed a procedure that uses a three-axis acceleration and rotation rate sensor to record longitudinal, lateral, and vertical acceleration and roll, pitch, and yaw rates in order to analyze the braking behavior of the vehicle in a dynamic test drive.

The main goal of this study is to classify whether a vehicle's braking system is intact or defective using sensor data from deceleration measurements series and using networks for that. Three different types of neural network architectures were trained and evaluated: a Fully Convolutional Network (FCN), a 2D Convolutional Neural Network (CNN), and a Long Short-Term Memory (LSTM) network. The objective was to determine which architecture performs best in predicting brake conditions for unknown vehicles.

In deep learning, the quality of the data plays a crucial role in the success of a model. Often, datasets need to be expanded, cleaned, or adjusted to maximize the model's performance. Additionally, data often needs to be transformed into a suitable format that the model can process. Methods like data preprocessing and data augmentation were applied.

Another issue that datasets may have is bias. Bias in a dataset refers to the presence of systematic errors or distortions that cause the data to inaccurately represent the true characteristics or diversity of the population it is supposed to reflect. So, to get a general model, bias needs to be reduced. To reduce bias in the dataset, a debiasing variational autoencoder was implemented.

Legend:

- Nikita Biswas
- Sebastian Garcia
- Daniel Useche Corchuelo
- Chaeran Yoo

2 Task 1: FCN, 2D CNNs vs. LSTMs - Parameter Tests and Comparison

The main objective of task 1 was to solve the classification problem by training three different models: a Fully Convolutional Neural Network, a 2D Convolutional Neural Network, and Long-Short-Term Memory network. In the next sections, there is a description of the architecture of each network. An analysis of the results is shown, which include a confusion matrix and metrics such as validation and test accuracy and F1 score. The results shown are using a random split and with 10 iterations.

2.1 Exploratory Data Analysis (Train vs. Test data set)

Exploratory Data Analysis (EDA) was conducted as a preliminary step to better understand the data and discrepancies between the train and test data set, which ultimately would impact the model performance. The main focus of the analysis revolved around finding imbalances in the labels and car characteristics, differences between train and test data set distribution, and noticeable differences between signals, based on different features.

2.1.1 Class Distribution

An exploratory analysis was conducted to examine the distributions of velocity, average deceleration, and labels within the training and test datasets. The variable *mass* was excluded from this analysis, as approximately 46.5% of the test dataset contained missing values. Figure -1 illustrates a noticeable difference between the velocity distributions of the datasets. Specifically, the test dataset exhibits higher velocity values compared to the training dataset.

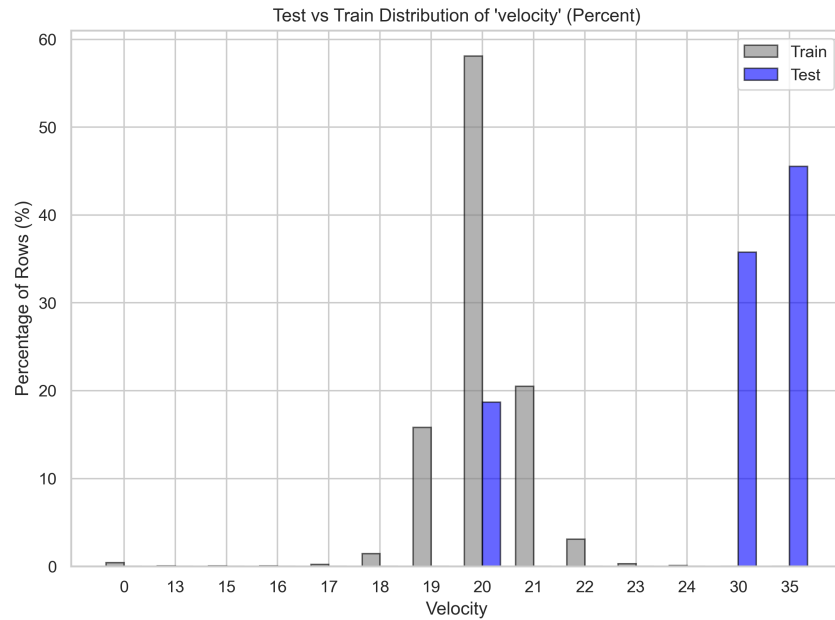


Figure -1: Comparison of Velocity Distributions (Train vs. Test)

In Figure -2, the distribution of average deceleration values clearly differs between datasets. The test data primarily clusters around values of $-6.5m/s^2$ and $-4m/s^2$, while the training data clusters predominantly around a central value of $-5m/s^2$, corresponding to the shallower portion of the test data distribution.

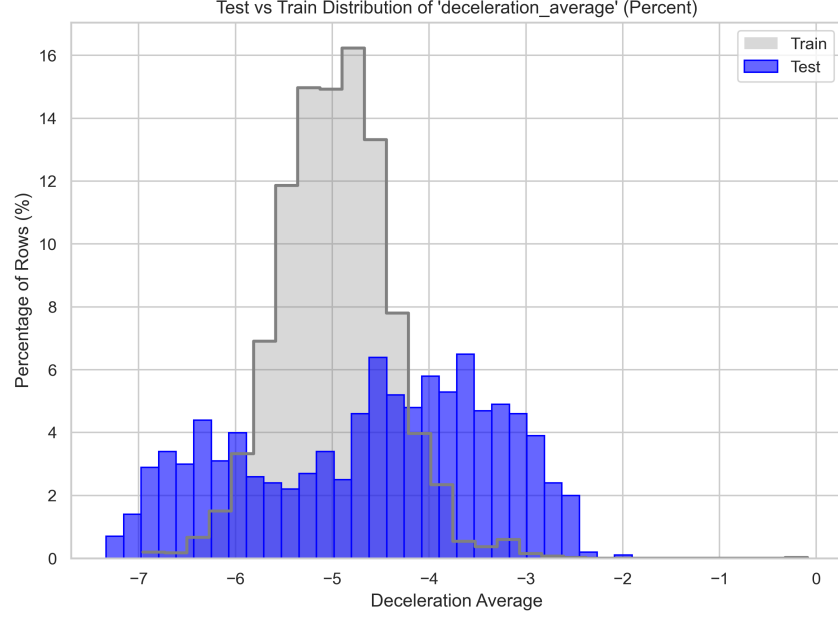


Figure -2: Comparison of Average Deceleration Distributions (Train vs. Test)

Finally, a pronounced class imbalance is observed, as shown in Figure -3. Approximately 45% of the training data and 60% of the test data belong to class label 0. The remaining classes are more evenly distributed across both datasets, except for labels (9, 10, 11), although only labels (1, 2, 4, 5) appear in the test dataset.

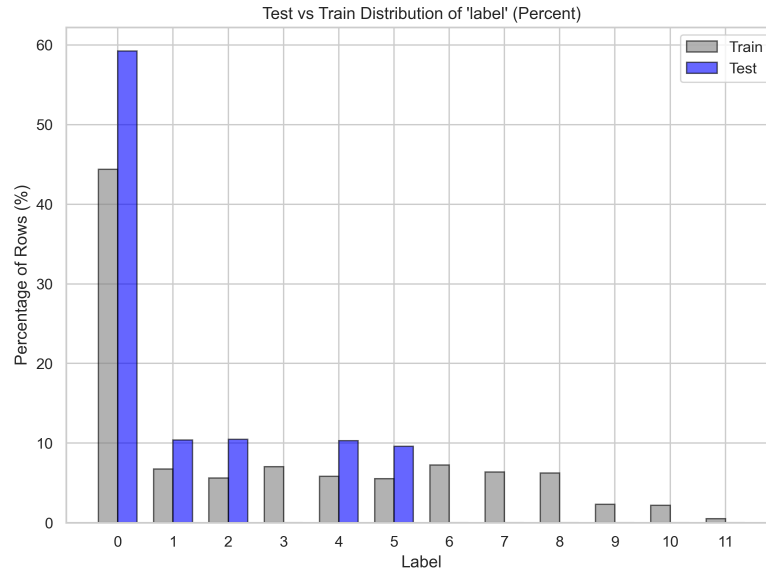


Figure -3: Comparison of Label Distributions (Train vs. Test)

2.1.2 Impact of Physical Features on Signals

Across different experiments, there is a clear pattern relating the amplitude of signals to certain physical features. Specifically, as mass, velocity, and the absolute value of the deceleration average increase, so does the amplitude of the signals. This trend is evident in both the training and test datasets, especially noticeable in the ax , az , and ry accelerations. Figures -4, -5, and -6 show examples illustrating this behavior.

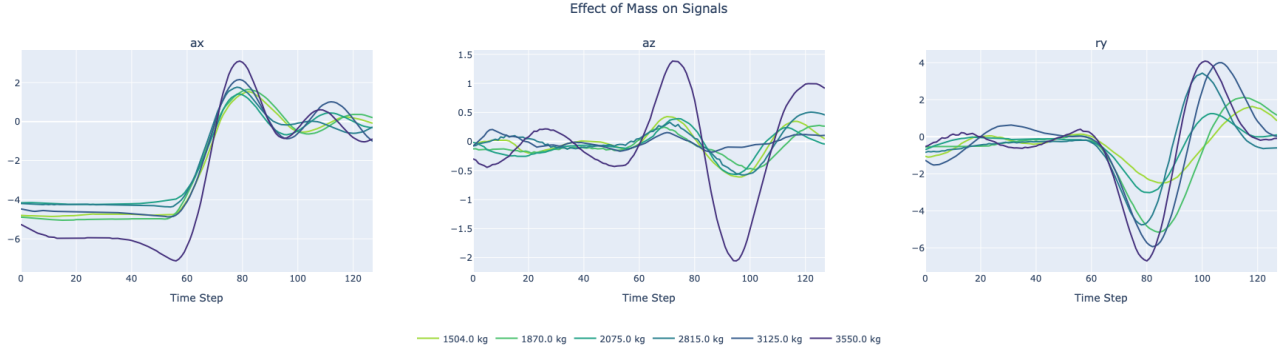


Figure -4: Impact of mass on the signals (Test Dataset)

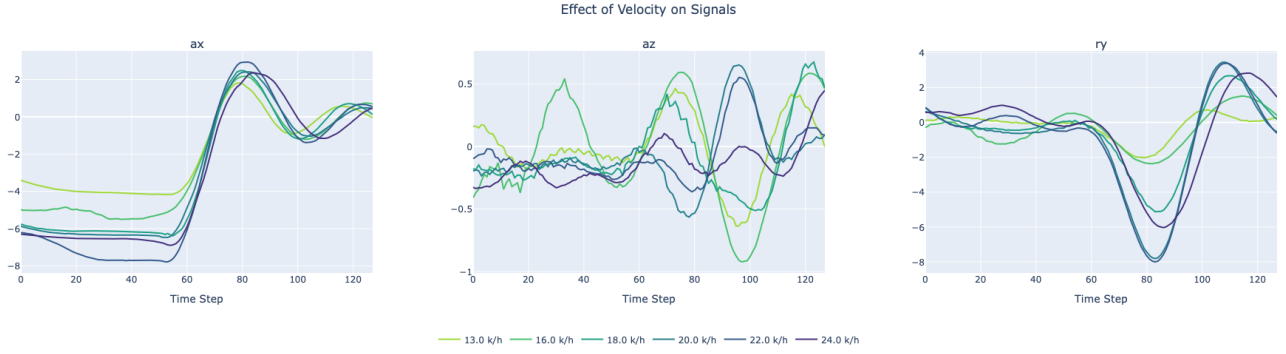


Figure -5: Impact of velocity on the signals (Train Dataset)

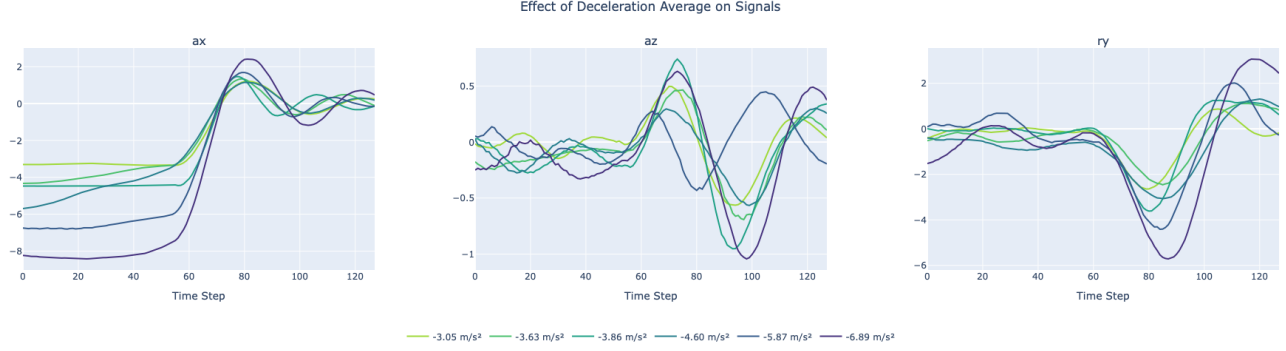


Figure -6: Impact of deceleration average on the signals (Test Dataset)

Some notable observations emerged during the analysis of the sensor data. A particularly interesting finding is the distinctive inverse behavior observed in the **az**-acceleration signals between the train and test datasets. This phenomenon is clearly illustrated in Figure -7.

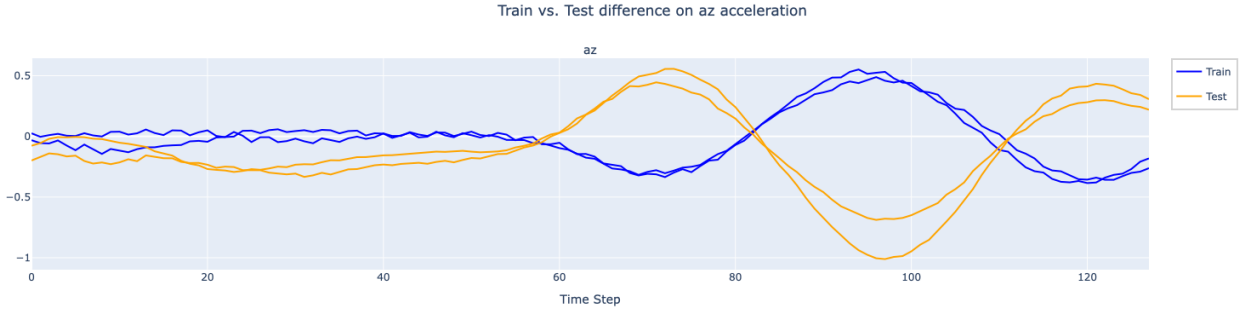


Figure -7: Inverse Pattern in az Acceleration between Train and Test Datasets

Another critical observation pertains to the disruptive nature of signals from the *forfour* model. This model comprises 46.5% of the test dataset and coincides with the instance with missing values in the mass. Signals from *forfour* exhibit significantly irregular patterns characterized by sudden spikes and drops, absence of a clear or consistent trend, and highly non-smooth, non-differentiable waveforms. Representative examples illustrating these erratic behaviors can be seen in Figure -8.

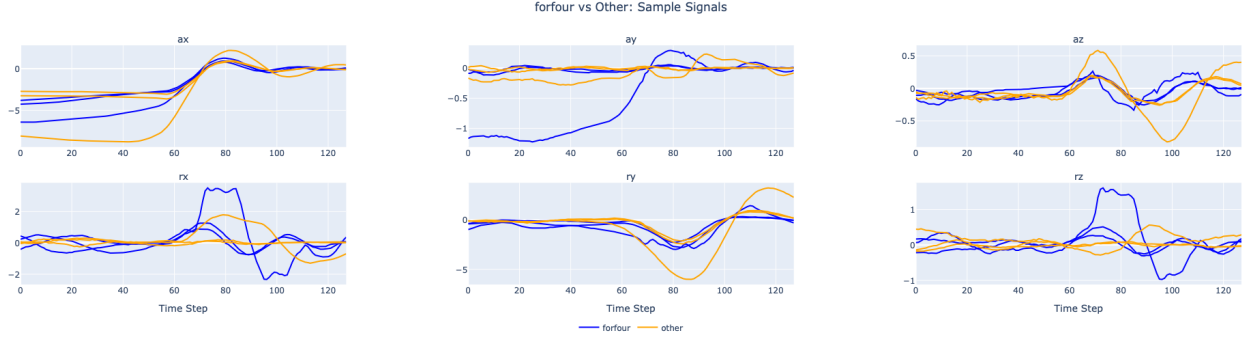


Figure -8: Irregular and Disruptive Signal Patterns of the *forfour* Model

2.2 FCN

2.2.1 Background

A Fully Convolutional Network (FCN) is a type of neural network that uses only convolutional layers (and sometimes pooling and normalization) without any fully connected (dense) layers at the end. Instead, it produces predictions by applying a final 1x1 convolution followed by global pooling or flattening.

For time-series data like deceleration measurements, an FCN processes the entire sequence in parallel using 1D convolutions to extract local and global temporal patterns across features such as accelerations.

Basically, a 1D FCN works by applying convolution filters across the time dimension, capturing patterns and correlations in how sensor signals evolve.

2.2.2 Architecture

The architecture used is a Fully Convolutional Network (FCN) designed for time-series classification of brake condition across 12 classes (labels 0–11). It consists of an input layer, three sequential 1D convolutional blocks, a global average pooling layer, and a final fully connected (linear) output layer.

Each convolutional block is designed to progressively extract more complex temporal features from the input signal. The blocks are defined as follows:

- **ConvBlock 1:**
 - Conv1D(in_channels=6, out_channels=128, kernel_size=8, padding=4)
 - BatchNorm1D(128)
 - ReLU()
- **ConvBlock 2:**

- Conv1D(128, 256, kernel_size=5, padding=2)
- BatchNorm1D(256)
- ReLU()

- **ConvBlock 3:**

- Conv1D(256, 128, kernel_size=3, padding=1)
- ReLU() (no BatchNorm here)

After the convolutional layers, a `GlobalAveragePooling1D` layer reduces the temporal dimension by averaging each feature map over time. This results in a fixed-size feature vector independent of the sequence length.

The final classification is performed using:

- `Linear(in_features=128, out_features=12)`

Functional Summary:

- The first convolutional block captures local short-term patterns in the signal (e.g., initial deceleration features).
- The second block abstracts mid-level temporal dependencies.
- The third block focuses on high-resolution or fine-grained signal variations.
- The global average pooling ensures a fixed-length output and helps reduce overfitting.
- The final dense layer performs the classification.

Regularization and Stability: Batch normalization is applied after the first two convolutional layers to stabilize and accelerate training. ReLU is used as the activation function to introduce non-linearity.

2.2.3 Analysis and Results

The tuning of the hyperparameters was done in the training and validation step. The hyperparameters that showed the best results for the FCN are shown in Table -1.

Table -1: Hyperparameters used for the Fully Convolutional Network (FCN)

Hyperparameter	Value
Batch size	32
Learning rate	1×10^{-4}
Number of epochs	20
Optimizer	Adam
Activation function	ReLU
Kernel size	8 ,5 and 3
Output channels (Conv1)	64
Output channels (Conv2)	128
Output channels (Conv3)	128

The main results include the average test accuracy, the macro F1 score, and the validation accuracy over 10 random splits. These are shown in Table -2.

Table -2: Performance metrics of the Fully Convolutional Network (FCN) over 10 random splits

Metric	Value
Average Test Accuracy	0.4840
Average Macro F1 Score	0.3401
Average Validation Accuracy	0.9911

Additionally, a confusion matrix is presented in Figure -9 for the split that achieved the highest accuracy.

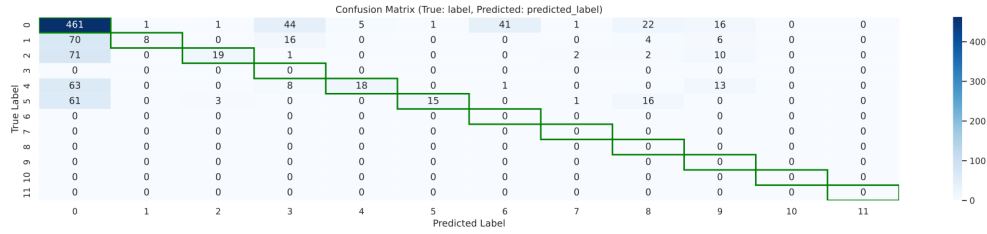


Figure -9: Confusion Matrix from the FCN

The confusion matrix reveals that the model has a strong bias toward predicting class 0, which shows a high number of correct predictions compared to the other classes. A substantial number of instances from several other classes (e.g., class 1, 2, 4, and 5) are misclassified as class 0, indicating either a significant class imbalance or overfitting dominant class features.

Furthermore, some classes such as 6 to 11 are never correctly predicted, which highlights the model’s inability to distinguish between minority classes. This is also reflected in the low macro F1 score, suggesting that performance is skewed and not evenly distributed across all classes.

To mitigate this issue, future work should consider applying data balancing techniques such as class-weighted loss functions, oversampling of underrepresented classes, or advanced debiasing methods. Evaluating with macro-averaged metrics remains essential to ensure fair performance across all classes.

2.3 2D CNN

2.3.1 Background

The 2D Convolutional Neural Network (2D CNN) is a well-established architecture for learning hierarchical patterns from structured input data. In this work, we apply a 2D CNN to sensor-based time-series data, where each sample is represented as a 2D array with the vertical axis corresponding to time steps and the horizontal axis representing multiple sensor channels.

Unlike 1D CNNs that process each channel independently across time, the 2D CNN enables the model to learn both temporal dynamics and inter-sensor dependencies simultaneously. By using a kernel of size 1×6 , for instance, the convolution layer focuses on the full set of sensor channels at each time step, effectively treating the input as a narrow "image" with height (time) and width (sensor features). This approach allows the model to discover correlations between sensor modalities without collapsing the temporal structure.

2.3.2 Architecture

The CNN model consists of two convolutional blocks followed by fully connected layers for classification. The model receives 2D input in the shape of (1, time steps, sensor channels), treating each sample as a narrow image.

- The first convolution layer applies a kernel of size (1×6) , which spans all sensor channels at a single time step. This allows the network to learn inter-sensor dependencies. It is followed by batch normalization, ReLU activation, max pooling along the time axis, and dropout.
- The second convolution layer uses a (3×1) kernel to capture short-term temporal patterns across consecutive time steps while preserving the sensor dimension. This is again followed by batch normalization, ReLU, pooling, and dropout.

The resulting feature map is flattened and passed through a fully connected layer, then a final output layer with 12 units for classification.

All intermediate layers use ReLU activations, and dropout is applied at multiple stages to mitigate overfitting. The architecture is modular and controlled via tunable hyperparameters (e.g., number of channels, hidden size, dropout rate).

2.3.3 Result and Analysis

Table -3 shows the best hyperparameter configuration identified during our experiments with the 2D CNN architecture. Table -4 presents the average validation and test performance over 10

Table -3: Best Hyperparameter Configuration for 2D CNN

Hyperparameter	Value
Batch size	64
Conv1 out channels	32
Conv2 out channels	64
Dropout rate	0.01
FC hidden units	64
Learning rate	0.001
Weight decay	0
Training iterations	100

runs using the best configuration. Early stopping with a patience of 10 epochs is applied to prevent overfitting. The validation performance was consistently high, indicating that the model

successfully fit the training data. However, a noticeable gap in test performance suggests a potential overfitting issue or the presence of dataset bias.

Table -4: Performance Metrics (Average of 10 Runs)

Metric	Value
Average Validation Accuracy	0.9975
Average Validation F1 Score	0.9971
Average Test Accuracy	0.4390
Average Test F1 Score	0.3898

To further understand the classification behavior, we analyzed the confusion matrix from the best-performing run (Figure -10). The model classifies frequent classes like class 0 well but struggles with rare or similar classes. Many misclassifications involve confusion with class 0 or class 9. This suggests the need for data augmentation to improve performance on underrepresented classes.

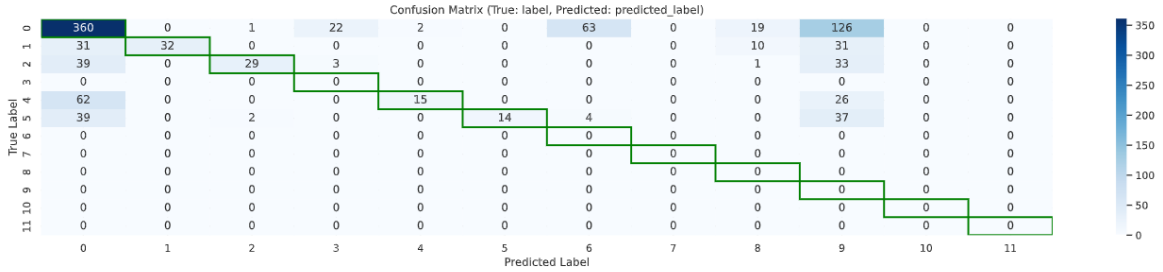


Figure -10: Confusion matrix of the best-performing 2D CNN Model

2.4 LSTM (Long Short-Term Memory)

2.4.1 Background

The development of Long Short-Term Memory (LSTM) networks is rooted in the hindrance of the ability of Recurrent Neural Networks (RNN) to learn long-term dependencies by the vanishing and exploding gradient problems. An LSTM is a modified RNN that replaces each simple recurrent unit with a memory cell. This memory cell maintains an internal state across time steps via a self-loop with a fixed weight, allowing gradients to propagate over long sequences without decay or explosion. The term “long short-term memory” refers to the ability of the model to learn both long-term patterns through the memory cell and short-term dynamics through transient activations.

For sequence classification, a compact Long-Short Term Memory (LSTM) network that processes each brake-test trace as a tensor of shape (128 timesteps $\times$ 6 sensor channels) was employed. The sequence is fed unchanged in length into a batch-first LSTM stack. At every time-step the LSTM updates its gated cell state, allowing it to preserve transient signatures such as a yaw-rate spike or sustained deceleration across dozens of steps without suffering vanishing gradients. When the

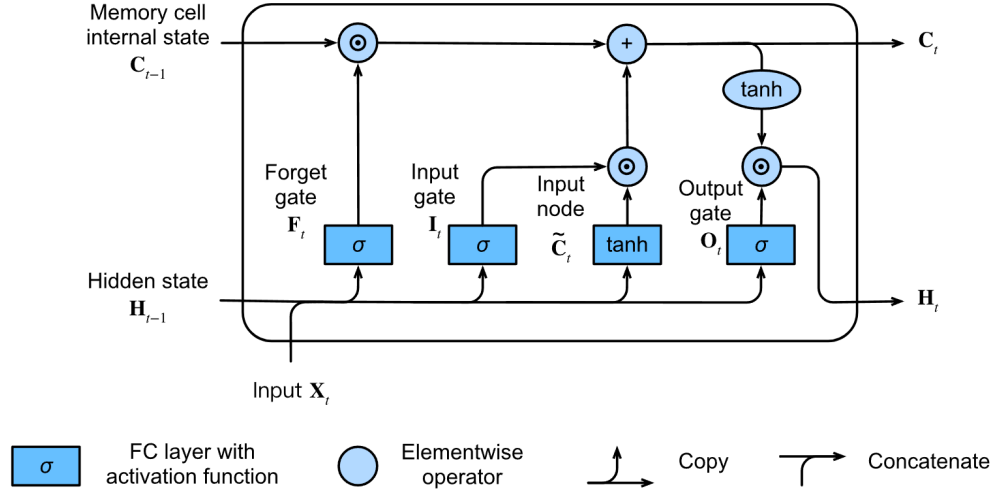


Figure -11: Visual depiction of the LSTM architecture

final timestep is reached we extract the last hidden state, producing a multi-dimensional summary vector. A single fully-connected layer then maps this summary to 12 class logits, one for each brake-condition pattern. Training minimises cross-entropy between these logits and the ground-truth label, so the model learns to transform raw multivariate acceleration/gyro curves into a discriminative temporal embedding that reflects the underlying health of the vehicle's braking system.

2.4.2 Architecture

A Long Short-Term Memory (LSTM) network is well suited to the requirement of capturing both short-term bursts and longer-range context:

- **Gate-controlled memory:** The input, forget, and output gates allow the network to preserve key moments, such as the onset of a brake anomaly, across many time steps while discarding irrelevant noise.
- **Resistance to vanishing gradients:** The self-connected memory cell passes information through the 128-step sequence without degrading, enabling the model to link early-sequence cues (e.g., initial pedal press) to late-sequence outcomes (e.g., residual yaw).
- **Fixed-length trace to fixed-size embedding:** By reading the final hidden state (or an attention-pooled state in later variants), the LSTM compresses a variable-complexity time course into a compact vector suitable for a soft-max classifier.

2.4.3 Analysis

In order to establish a baseline model, extensive experimentation was carried out by creating various combinations of hyperparameters. Out of those experiments, the hyperparameters and results from the best performing model have been tabulated below.

Hyperparameter	Value
Batch size	64
Learning rate	1×10^{-3}
Number of epochs	40
Optimizer	Adam
Number of layers	4

Table -5: Baseline LSTM model hyperparameters

The aforementioned hyperparameters were utilized over 10 random splits resulting in the following averaged results.

Metric	Value
Average Test Accuracy	0.3721
Average Macro F1 Score	0.2165
Average Validation Accuracy	0.9973

Table -6: Performance metrics of LSTM over 10 random splits

The best experiment yielded the following confusion matrix. Figure -12 indicates that the model has learned to detect “no fault” well but cannot distinguish among the 11 faulty-brake patterns. Moreover, many mis-labeled “0”s are still guessed as other classes (e.g. labels 3, 4, 5, 6), but almost no true faults (classes 7–11) are ever recovered as those rows are entirely zero. As evident from the low F1-score, rare classes are almost never predicted correctly; the model effectively collapses onto the most frequent labels.

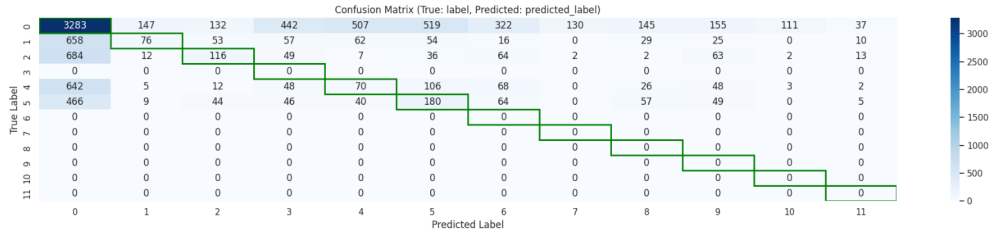


Figure -12: Confusion Matrix from the LSTM

Further steps to improve this model could involve handling class imbalance by oversampling of the minority class or down-sampling the majority class.

2.5 Best Network

The network that achieved the best accuracy and best F1 score was the FCN with an average test accuracy of 48% and an average Macro F1 score of 34%. The model does not effectively distinguish between most brake conditions. It shows that it is underfitting for minority classes or overfitting to class 0. It is likely due to the severe class imbalance in the training set.

In the next task, these issues are going to be addressed through pre-processing techniques, data augmentation and finally by implementing a debiasing variational autoencoder in order to minimize bias toward class 0.

3 Task 2: Bias in the dataset

The main objective of task 2 is to improve the quality of the training dataset through preprocessing and data augmentation techniques in order to maximize the model's performance. Also, we saw that there is a bias towards class 0, so to overcome this techniques such as oversampling or more complex ones like implementing a debiasing variational autoencoder are employed.

3.1 Network's performance

In this section, a performance analysis of the metadata is shown. The following graphics show how high is the accuracy by model variant (Figure -13), by mass (Figure -14), by velocity (Figure -16), and by deceleration average (Figure -15).

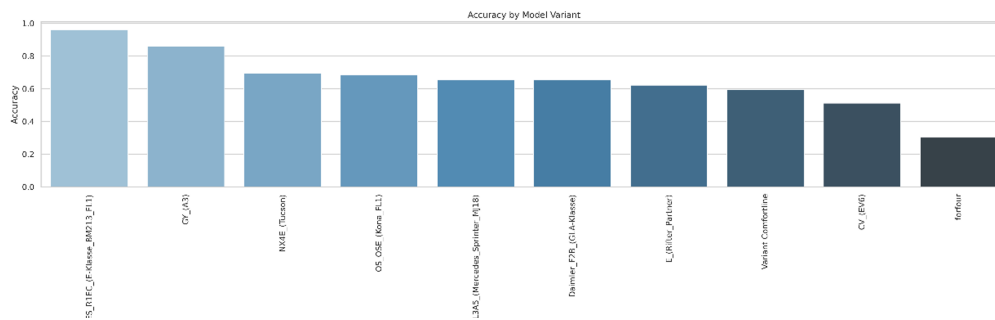


Figure -13: Accuracy by model

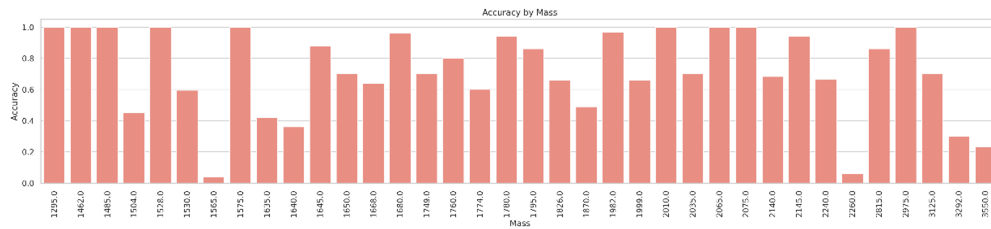


Figure -14: Accuracy by mass

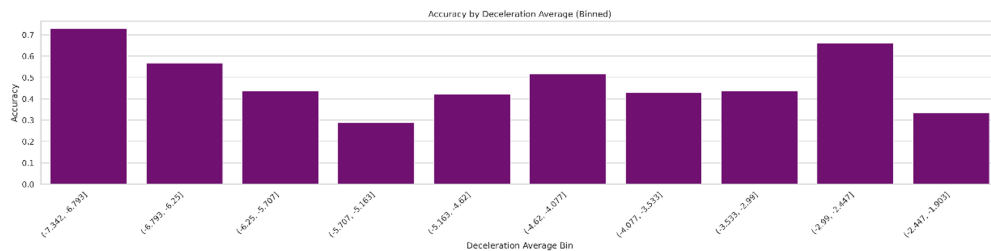


Figure -15: Accuracy by deceleration average

The following conclusions were drawn:

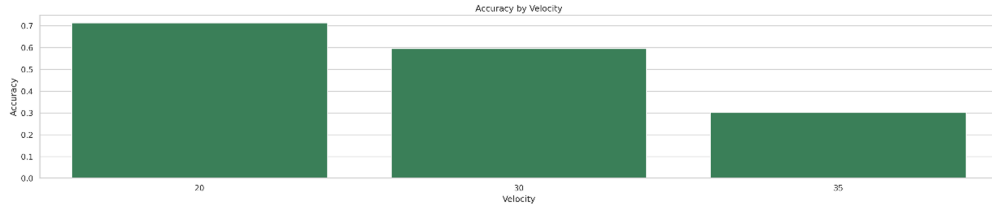


Figure -16: Accuracy by velocity

- The model exhibits high accuracy for some vehicle variants, such as *Daimler_212* and *GY*, but performs poorly on others like *forfour* and *CV*. This disparity suggests a class imbalance or insufficient representation of certain variants during the training process.
- Accuracy by mass shows substantial variability across different vehicle mass values. Some mass ranges (e.g., around 1550 kg) have particularly low accuracy, likely indicating a correlation between mass and specific underrepresented.
- Accuracy by velocity suggests that the model struggles to generalize well under high-speed conditions.
- The model shows uneven performance across different deceleration bins, with some bins yielding significantly lower accuracy. These patterns may correspond to specific braking behaviors that the model finds difficult to classify.

3.1.1 Deceleration measurements

In the FCN model, we tested which deceleration measurements performed well and which ones were suboptimal. The a_x , a_y deceleration measurements, and the roll rate dx were removed, and the model had an average test accuracy of 0.5118 over 10 random splits, with an average test F1 score of 0.2994. This means the model performs well when the a_z , dy , and dz measurements are used. When these measurements are removed, the model does not perform well. The average test accuracy over 10 random splits was 0.3509, meanwhile, the average test F1 score over 10 random splits was 0.1703.

3.2 Data preprocessing

Two techniques were used: standardization and min-max scaling. In the case of FCN, the method of standardization didn't improve the overall performance of the model. The average test accuracy over 10 random splits was 0.4934 and the average test F1 score over 10 random splits was 0.3188. On the other hand, normalization didn't work at all, and the accuracy test dropped big amounts.

3.3 Data augmentation

Data augmentation is a technique used to artificially expand your training dataset by creating modified versions of existing data sample. The reasons to use data augmentation are various:

- Improve generalization.
- Handle imbalanced classes.
- Increase dataset size

In this scenario, the experiments were focused on reducing the imbalance in classes and improving the robustness compared to the baseline model. All experiments were conducted for 20 epochs over 10 random splits. The techniques utilized and their corresponding findings are listed below:

- **Gaussian noise injection:** Adds zero-mean Gaussian noise to every sensor channel. The technique tests the robustness of the model to sensor jitter and measurement error. Moderate noise (0.005–0.01) resulted in better test accuracy (0.49), but overly small or large noise degraded the performance.

Noise	Test accuracy	Macro F1 score (test)
0.001	0.4693	0.3134
0.005	0.4739	0.3139
0.01	0.4939	0.3067
0.02	0.4627	0.3128

Table -7: Experimentation results over 10 random splits with Gaussian noise

- **Time shifting:** Simulates slight misalignment in measurements. The objective was to gauge if the model is less sensitive to alignment. Small shifts (± 5) slightly boosted test accuracy but larger shifts began to harm F1, suggesting too much misalignment confuses the model.

Parameter (shift)	Test accuracy	Macro F1 score (test)
2	0.4702	0.3149
5	0.4770	0.2973
10	0.4654	0.3015

Table -8: Experimentation results over 10 random splits with Time shift

- **Scaling:** Varies the signal amplitude and simulates differences in sensor calibration or brake intensity. In this technique, we scaled specific axes by a random factor sampled from a range. Additionally, two types of range were chosen: Short range (0.8,1.2) and extreme range (0.5,2.0). Tight scaling on the two most critical channels yielded better F1 score, whereas extreme scaling ranges could hurt accuracy.

Axes	Scaling range	Test accuracy	Macro F1 score (test)
3,4	(0.8,1.2)	0.4757	0.3231
3,4	(0.5,2.0)	0.4578	0.2944
2,4,5	(0.5,2.0)	0.4563	0.3256

Table -9: Experimentation results over 10 random splits with scaling

- **Truncation:** Remove a fixed fraction (e.g. 10-30 %) of the beginning of each signal. The technique tests whether early-segment information is redundant or misleading. Even modest truncation eroded performance, confirming that early-phase data contain useful cues.

Truncation %	Test accuracy	Macro F1 score (test)
10	0.4601	0.2959
30	0.4205	0.2546

Table -10: Experimentation results over 10 random splits with Truncation

- **Sliding window:** Split each signal into smaller windows (e.g. 32 or 64 timesteps) with no overlap, treats each window as an independent training example. This technique allowed the model to focus on local temporal patterns. Windows of length 32 resulted in better test accuracy, but degraded the F1-score.

Window length x Stride	Test accuracy	Macro F1 score (test)
32 x 16	0.4921	0.2496
64 x 32	0.4521	0.2515

Table -11: Experimentation results over 10 random splits with sliding

- **Selective augmentation:** In order to gauge the impact of mixed augmentation techniques, selected minority classes were over-sampled and moderate noise was introduced. The average test accuracy was 0.4851 and test F1-score was 0.3084. Since the oversampling of minority classes did not yield in a significant improvement in accuracy, an additional experiment was carried out by downsampling the majority class (Label 0). However, the model could still not separate between amplitude and timing nuances and the likely reason being that labels 3 and 6 share early-sequence shape with label 0.

To oversample the underrepresented classes a function was created to do selective augmentation based on conditions, such as labels, models, velocities, etc.

Another way of doing the data augmentation was to increase the number of unrepresented classes by applying data augmentation to measurements that had labels different from zero. So, for each record with a label different from zero, we applied Gaussian noise, temporal shift, and scaling at the same time. The original distribution of labels is shown in Table -12:

Table -12: Label distribution in the training dataset

Label	Number of Samples
0	1798
1	273
2	227
3	286
4	236
5	225
6	294
7	258
8	253
9	94
10	89
11	20

After the data augmentation, the following distribution was achieved, shown in Table -13:

Table -13: Label distribution in the training dataset with data augmentation

Label	Number of Samples
0	1798
1	1092
2	908
3	1144
4	944
5	900
6	1176
7	1032
8	1012
9	940
10	890
11	820

In this experiment, by applying data augmentation to underrepresented classes, an average test accuracy over 10 random splits of 0.4301 was obtained with an average macro F1 score of 0.1684.

3.4 Debiasing Variational Autoencoder

This work investigates whether adding a Debiasing Variational Autoencoder (DB-VAE) structure to a baseline FCN can improve generalization performance on a physical activity recognition task with imbalanced classes. The DB-VAE is designed to learn a more balanced latent representation by combining unsupervised reconstruction with supervised classification, while introducing KL divergence to regularize the latent space.

3.4.1 Model Architecture

The architecture consists of three main components: the encoder, decoder, and classifier.

1) Encoder (Feature Extractor): The encoder transforms an input multivariate time-series of shape $(\text{batch_size}, 128, 6)$ into a low-dimensional latent representation. It uses three 1D convolutional layers inspired by the FCN architecture:

- **Conv1D:** $6 \rightarrow 64$, kernel size = 8, padding = 4 $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- **Conv1D:** $64 \rightarrow 128$, kernel size = 5, padding = 2 $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- **Conv1D:** $128 \rightarrow 128$, kernel size = 3, padding = 1 $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- **Adaptive Average Pooling:** Output shape $\rightarrow (\text{batch_size}, 128, 1)$

The pooled output is flattened and passed through two fully connected layers to produce the latent distribution parameters: mean μ and log-variance $\log \sigma^2$. Latent vectors z are sampled using the reparameterization trick.

2) Decoder (Reconstruction Head): The decoder reconstructs the input from the latent vector z . A single linear layer maps $z \in \mathbb{R}^{16}$ to a vector of shape 128×6 , which is then reshaped into the original input format $(128, 6)$.

3) Classifier (Prediction Head): A fully connected layer maps the latent vector z to a 12-dimensional output, corresponding to the activity class logits. This classifier is trained jointly with the VAE.

4) Training Objective: The total loss $\mathcal{L}_{\text{total}}$ consists of three components:

$$\mathcal{L}_{\text{total}} = \text{MSE} + \beta \cdot \text{KL} + \lambda \cdot \text{CE}$$

- **Reconstruction Loss (MSE):** Mean squared error between original and reconstructed inputs.
- **KL Divergence (KL):** Regularizes the latent space toward a standard Gaussian.
- **Classification Loss (CE):** Cross-entropy between predicted and ground-truth labels.

3.4.2 Result and Analysis

Table -14 presents the average results of 10 runs. Early stopping with a patience of 10 epochs is applied to prevent overfitting. The FCN-DBVAE model achieved a test accuracy of **48.61%** and a macro F1 score of **35.44%**, slightly outperforming the baseline FCN classifier.

Model	Test Accuracy (%)	Macro F1 (%)
FCN Classifier (Baseline)	48.40	34.01
FCN-DBVAE	48.61	35.44

Table -14: Comparison of test set performance (average over 10 runs).

Although the improvements are marginal, they indicate that incorporating variational regularization and auxiliary reconstruction may help the model learn more robust representations, especially under class imbalance. The best performance was observed with the following hyperparameters.

Hyperparameter	Value
Batch size	64
Learning rate	1×10^{-3}
Latent dimension	16
KL weight (β)	1×10^{-3}
Classification loss weight (λ)	1.0

Table -15: Hyperparameter settings for the best-performing DB-VAE model

These settings struck a good balance between reconstruction quality and classification performance. Larger values of β tended to degrade classification accuracy, possibly due to excessive regularization and posterior collapse.

We additionally tested a configuration with a smaller batch size (32) and a lower learning rate (1×10^{-4}). This setting yielded a slightly higher average accuracy of 50.63%, but the average macro F1 score dropped significantly to 26.12%. This result suggests that the model likely overfitted to majority classes, achieving high accuracy at the cost of per-class balance, indicating stronger class imbalance issues under this setup.

These findings motivate future work in two directions: improving the variational architecture to better utilize the latent space for classification, and incorporating advanced data rebalancing techniques during training. Further investigation into more robust and disentangled latent representations could potentially yield stronger gains in both accuracy and fairness.

4 Additional Task 2: Ensemble Classifiers

In this section, two ensemble strategies built on the base sequence models (LSTM, CNN, and FCN) were explored. The first approach is a *Binary Head Model*, where a single network jointly predicts a binary label indicating whether the brakes are *working* (0) or *malfunctioning* (1) and a multiclass label for the specific failure type, but only for those instances flagged as malfunctioning. This approach intends to create an specialized model for the malfunctioning types classification, instead of a general model.

The second approach is a *Physical Features Head Model*, which appends an auxiliary head that attempts to predict the physical attributes (mass, velocity, and deceleration average) alongside the brake-status classification. The intuition is that forcing the network to learn these correlated features may yield deeper representations of the sensor signals.

4.1 Binary Head Model

Figure -17 shows the overall architecture of the model. Rather than training two separate models and cascading their predictions (which would amplify errors), both tasks share a common feature extractor and are optimized jointly. A weighting factor λ balances the binary and multiclass losses in the total loss.

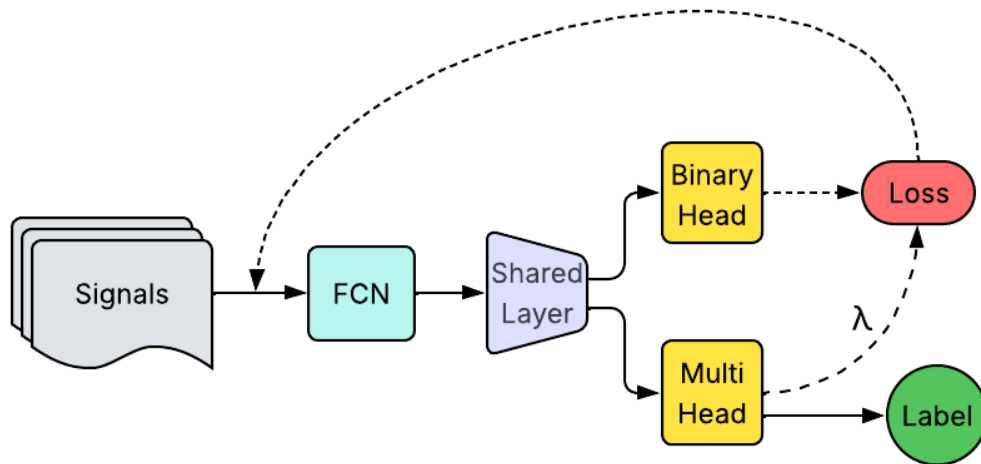


Figure -17: Architecture of the Binary Head Model with shared feature extractor and two prediction heads.

Although its overall test accuracy did not exceed our best single-task models, the *Binary Head Model* achieved a higher macro-F1 on the malfunctioning class, demonstrating improved detection of brake failures at the expense of some multiclass performance. Table **-16** summarizes these key results.

Metric	Value
Iterations	20
Test Accuracy	0.4319
Test macro-F1 Score	0.4047

Table -16: Performance metrics for the Binary Head Model.

In Table -17, there is a summary of the data augmentation steps and core hyperparameters used to train the model. Interestingly, applying oversampling to the working-brake class, improved performance on the malfunctioning classes.

Hyperparameter	Value
Learning rate	1×10^{-3}
Number of epochs	5
λ (loss weight)	1
Oversampling target	labels = 0
Oversample multiplier	1
Standardize	True
Scaling range	(0.8, 1.8)
Scaled channels	ax, az, ry

Table -17: Data augmentation and training hyperparameters for the Binary Head Model.

Figure -18 displays the confusion matrix on the test set for the model. Compared to the earlier single-task classifiers, this model more accurately detects brake malfunctions, as evidenced by the higher true-positive rates and reduced off-diagonal errors in the malfunctioning classes. However, the model still falls short of truly generalizing on the test dataset.

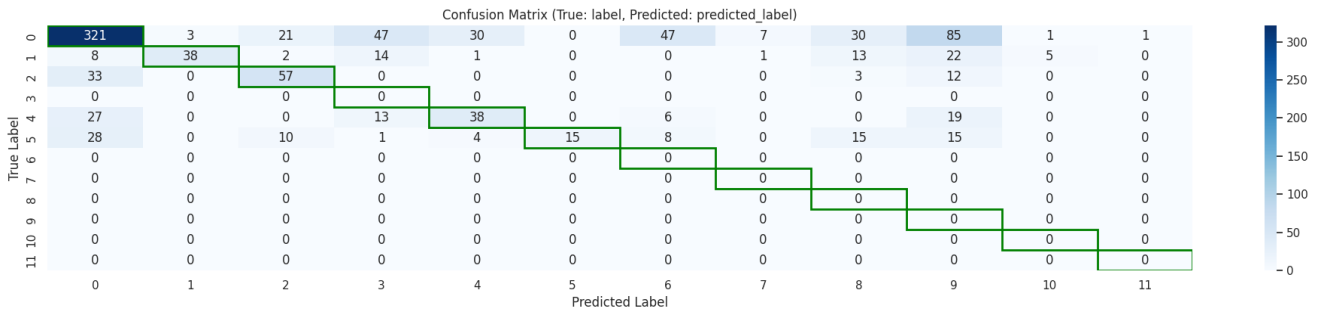


Figure -18: Confusion matrix of the Binary Head Model on the test dataset.

4.2 Physical Features Head Model

Motivated by our EDA, which revealed systematic shifts in signal amplitude with mass, velocity, and average deceleration, instead of explicitly modeling these relationships, we extended the base sequence model with an additional regression head designed to predict these three physical features

directly from the raw sensor data. The architecture, shown in Figure -19, again uses a shared backbone and weight parameter λ to trade off classification vs. physical features predictions.

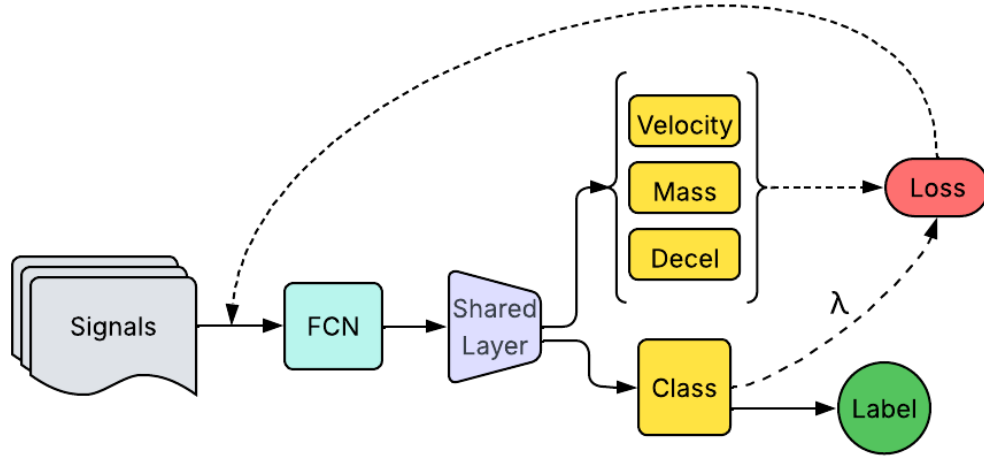


Figure -19: Architecture of the Physical Features Head Model: a shared encoder supports both classification and regression tasks.

While it was hypothesized that incorporating physical regression would improve feature extraction and overall performance, this model did not outperform the best standalone classifier introduced in section 3. This poor performance is likely due to a mismatch in the distributions of the physical features between the training and test sets, limiting the regression head's ability to generalize.

5 Conclusion

The classification of deceleration measurement sequences in real-world scenarios remains challenging, primarily due to the pronounced class imbalance in the dataset. This imbalance inherently limits the maximum achievable accuracy, as models tend to favor the majority class and struggle to generalize to underrepresented vehicle types.

In an initial comparison of three architectures (Fully Convolutional Network (FCN), 2D Convolutional Neural Network (CNN), and Long Short-Term Memory (LSTM)), the FCN exhibited the best overall performance on the test set. Its ability to extract localized temporal features with relatively low complexity resulted in more stable and efficient training. In contrast, LSTMs tended to overfit due to their larger parameter space, while 2D CNNs failed to naturally preserve the sequential structure of the deceleration data.

To address the imbalance, several preprocessing and augmentation techniques were applied. However, these methods had negligible effect on test accuracy and F1 score, suggesting that augmenting existing samples cannot compensate for the lack of truly representative measurements of rare classes. Augmentation increased robustness but did not introduce new patterns necessary for learning unseen vehicle models.

Subsequently, a Debiasing Variational Autoencoder (DB-VAE) was explored to learn a more disentangled latent space. The DB-VAE achieved a test accuracy of 48.61% and a macro F1 score of 35.44%, marginally surpassing the FCN baseline (48.40% accuracy, 34.01% F1). With the batch size of 32 and learning rate 1×10^{-4} , accuracy rose to 50.63%, but macro F1 fell to 26.12%, emphasizing the model’s struggle to generalize across minority classes. These outcomes underscore the difficulty of mitigating real-world imbalance without stronger debiasing objectives or architectural innovations.

Task-specific “heads” attached to the base network were also evaluated. A binary classification head focusing on malfunctioning brakes improved detection within that subgroup, yielding a higher macro F1 score of 40.47% though overall accuracy remained limited at 43.19%. A physics-informed head, designed to account for vehicle mass and braking-test conditions, showed theoretical promise but was impeded by the narrow range of physical parameters in the training data. A richer dataset encompassing diverse masses and test scenarios would be necessary to validate this approach.

Future work should focus on expanding the dataset with additional classes and richer contextual metadata (e.g., road friction, brake-pad wear) to achieve meaningful generalization. Preliminary ensemble experiments hint that hybrid architectures may yield further gains. In particular, Yang et al. (2) demonstrated that a hybrid LSTM–CNN–Attention model for braking-intention prediction outperformed pure LSTM and CNN baselines by 3.16%, with an additional $\sim 2\%$ improvement after genetic-algorithm hyperparameter optimization. Such hybrid and automated tuning strategies warrant investigation within the deceleration classification framework.

References

- [d2l] 10.1. Long Short-Term Memory (LSTM) &x2014; Dive into Deep Learning 1.0.3 documentation — d2l.ai. https://d2l.ai/chapter_recurrent-modern/lstm.html. [Accessed 12-06-2025].
- [2] Wei Yang, Yu Huang, K. J. Z. Z. K. Z. and Ruan, Q. (2024). Method of Predicting Braking Intention Using LSTM-CNN-Attention With Hyperparameters Optimized by Genetic Algorithm - International Journal of Control, Automation and Systems, link.springer.com. <https://link.springer.com/article/10.1007/s12555-021-1113-x>. [Accessed 12-06-2025].